Chapter THREE

Selection and Reptation

Making Decisions Using the if Statement

- The if and if-else statements are the two most commonly used decision-making statements in C#
- The if statement takes the form

```
if(expression)
    statement;
```

- Expression represents any C# expression that can be evaluated as true or false
- Statement represents the action that will take place if the expression evaluates as true

Making Decisions Using the if Statement

- Suppose we are building an ATM system where a user wants to withdraw money.

```
double balance = 1000;
double amount = 500;

if (amount <= balance)
    Console.WriteLine("Withdrawal successful");
```

- The expression **amount <= balance** checks if the user has enough balance.
- If it is **true**, the withdrawal is allowed.

What happens when Insufficient balance

Making Decisions Using the if Statement

- You can place any number of statements within the block contained by curly braces, including another if statement (which is called a nested if statement)
- In C#, you can place multiple statements inside a block using curly braces { }. These blocks can also contain another if statement, which is known as a nested if statement.

ATM Scenario Example (Nested if)

Suppose an ATM system first checks whether the user has enough balance, and then checks whether the withdrawal amount is within the daily limit.

```
double balance = 2000;
double amount = 500;
double dailyLimit = 1000;

if (amount <= balance)
{
    if (amount <= dailyLimit)
    {
        Console.WriteLine("Withdrawal successful");
    }

}
```

Explanation:

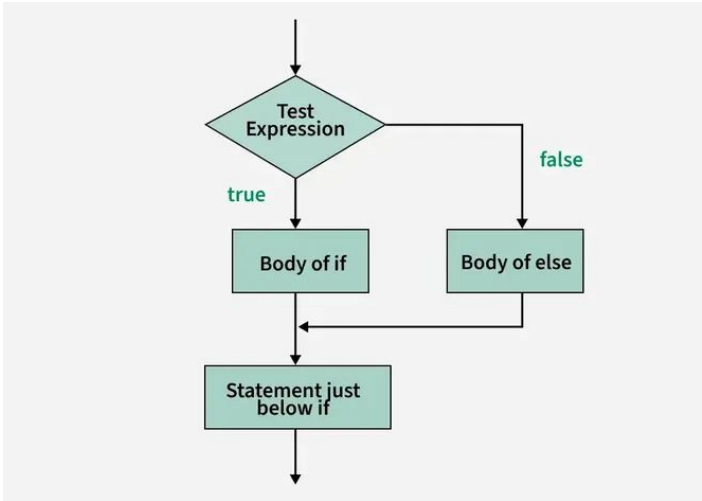
- The first if checks if the balance is sufficient.
- The second (nested) if checks the daily withdrawal limit.
- Both conditions must be satisfied for a successful transaction.

How do you inform the user whether they have reached the daily limit or have insufficient balance?

Making Decisions Using the if-else Statement

- Some decisions are dual-alternative decisions, meaning they have only two possible outcomes. In C#, the if-else statement is used to handle such situations. It takes the form:
- The if-else statement takes the form

```
if(expression)
        statement1;
else
        statement2;
```



Making Decisions Using the if-else Statement

If the expression evaluates to **true**, statement1 is executed.
Otherwise, statement2 is executed.

```
double balance = 800;
double amount = 1000;

if (amount <= balance)
    Console.WriteLine("Withdrawal approved");
else
    Console.WriteLine("Insufficient funds");
```

Explanation:

- The expression amount <= balance checks if the user has enough money.
- If true, the withdrawal is approved.
- If false, the system displays an insufficient funds message.

Making Decisions Using the if-else Statement

```
using System;

class ATM
{
   static void Main(string[] args)
   {
      double balance = 800;
      double amount;

      Console.WriteLine("=== ATM
System ===");
      Console.WriteLine("Your
current balance is: " + balance);
      Console.Write("Enter amount
to withdraw: ");
      amount =
Convert.ToDouble(Console.ReadLin
e());
      if (amount <= balance)
      {
         balance = balance - amount;
         Console.WriteLine("Withdrawal approved");
         Console.WriteLine("Remaining balance: " +
balance);
      }
      else
      {
         Console.WriteLine("Insufficient funds");
      }
      Console.ReadLine();
   }
}
```

Example Run

```
=== ATM System ===
Your current balance is: 800
Enter amount to withdraw: 500
Withdrawal approved
Remaining balance: 300
```

Another Example

```
=== ATM System ===
Your current balance is: 800
Enter amount to withdraw: 1000
Insufficient funds
```

C# Ternary (?:) Operator

Ternary operator are a substitute for if...else statement. So before you move any further in this tutorial, go through C# if...else statement (if you haven't).

The syntax of ternary operator is:

```
Condition ? Expression1 : Expression2;
```

The ternary operator works as follows:

- If the expression stated by `Condition` is `true`, the result of `Expression1` is returned by the ternary operator.

Rewrite the Above code using Ternary operator

Another Example

- If it is `false`, the result of `Expression2` is returned.

Using Compound Expressions In if Statements

- As an alternative to nested if statements, you can use the conditional AND operator within a Boolean expression to determine whether two expressions are true
- The two sample codes shown below work the same way

```
double balance = 2000;
double amount = 500;
double dailyLimit = 1000;

if (amount <= balance && amount
<= dailyLimit)
{
    Console.WriteLine("Withdrawal
approved");
}
```

Explanation:

- The operator && ensures that **both conditions must be true**.
- If either condition is false, the statement will not execute.
- This approach simplifies the code and improves readability.

Note:

- Use **compound expressions (&&)** when conditions are simple and related.
- Use **nested if statements** when you need more detailed control or different messages for each condition.
- Overusing nested `if` statements can make code harder to read and maintain.

Using Compound Expressions In if Statements

- When to use ternary operator? Why is it called ternary operator?
- For example, we can replace the following code

```
if (number % 2 == 0)
{
      isEven = true;
}
else
{
      isEven = false;
  }
```

Ternary operator:

- `isEven = (number % 2 == 0) ? true : false ;`

```
using System;

namespace Conditional{

class Ternary

        {public static void Main(string[] args){

int number = 2;

bool isEven;

isEven = (number % 2 == 0) ? true : false ;

                        Console.WriteLine(isEven)

;}}}
```

Using Compound Expressions In if Statements

```
using System;

class ATM
{
static void Withdraw(double balance, double
amount)
{
if (amount <= balance)
{
balance = balance - amount;
Console.WriteLine("Withdrawal successful");
Console.WriteLine("Remaining balance: " + balance);
}
else
{
Console.WriteLine("Insufficient balance");
}
}
```

```
static void Deposit(double balance, double
amount)
{
balance = balance + amount;
Console.WriteLine("Deposit successful");
Console.WriteLine("New balance: " + balance);
}
```

Using Compound Expressions In if Statements

```
static void Main(string[] args)
{
double balance = 1000;

Console.WriteLine("ATM Menu:");
Console.WriteLine("1. Withdraw");
Console.WriteLine("2. Deposit");

Console.Write("Choose option: ");
int choice = Convert.ToInt32(Console.ReadLine());

if (choice == 1)
{
Console.Write("Enter amount to withdraw: ");
double amount =
Convert.ToDouble(Console.ReadLine());

// Calling the method
Withdraw(balance, amount);
}
else if (choice == 2)
{
Console.Write("Enter amount to deposit: ");
double amount = Convert.ToDouble(Console.ReadLine());

// Calling the method
Deposit(balance, amount);
}

Console.ReadLine();
}
}
```

Using Compound Expressions In if Statements

- Using compound expressions in if statements allows you to evaluate multiple conditions in a single line.
- You are not required to use the AND operator (&&), because nested if statements can achieve the same result.
- However, compound expressions often make code shorter and easier to read.
- When using &&, you must include a complete Boolean expression on both sides of the operator.
- A common mistake is writing an incomplete condition such as if (saleAmount > 1000 && < 5000), which is incorrect because each side of && must be a full expression.

Using Compound Expressions In if Statements

- If the first condition in an && expression is false, the second condition is not evaluated because the overall result is already false.
- The conditional OR operator (||) is used when an action should occur if at least one of the conditions is true.
- You can combine multiple && and || operators in a single expression, and when both are used together, the && operator has higher precedence.
- Parentheses can be used to control the order of evaluation and ensure the logic behaves correctly.

ATM Example Using Compound Expressions (&& and ||)

```
using System;

class ATM
{
   static void Main(string[] args)
   {
      double balance = 1500;
      double amount = 700;
      double dailyLimit = 1000;
      bool cardActive = true;
      bool isPinCorrect = true;
      if ((amount <= balance && amount <= dailyLimit)
      || (cardActive && isPinCorrect))
            { Console.WriteLine("Transaction approved");
            }
            else
            {
      Console.WriteLine("Transaction declined");}
            Console.ReadLine();
      }
}
```

What would happened if the parentheses were not used in the above if statement?

Transaction Validation Scenario

```
using System;

class Blockchain
{
static void Main(string[] args)
{
double senderBalance = 1000;
double transactionAmount = 300;
double networkFee = 10;
bool isVerified = true;
bool isValidSignature = true;
bool isNetworkCongested = false;

if ((senderBalance >= transactionAmount + networkFee && isVerified && isValidSignature) &&
!isNetworkCongested)
{
Console.WriteLine("Transaction added to blockchain");
senderBalance = senderBalance - (transactionAmount + networkFee);
Console.WriteLine("Remaining balance: " + senderBalance);
}
```

Blockchain Transaction Validation Scenario

```
else if (!isVerified || !isValidSignature)
{
Console.WriteLine("Transaction rejected: Verification failed");
}
else if (isNetworkCongested)
{
Console.WriteLine("Transaction delayed: Network congestion");
}
else
{
Console.WriteLine("Transaction failed: Insufficient balance or invalid conditions");
}

Console.ReadLine();
}
}
```

C# switch Statement

- Switch statement can be used to replace the if...else if statement in C#. The advantage of using switch over if...else if statement is the codes will look much cleaner and readable with switch.
- The syntax of switch statement is:

```
switch (variable/expression){

    case value1:
        // Statements executed if expression(or variable) = value1
        break;
    case value2:
        // Statements executed if expression(or variable) = value1
        break;
    ... ... ...
    ... ... ...
    default:
        // Statements executed if no case matches
```

C# switch Statement

The switch statement evaluates the `expression` (or `variable`) and compare its value with the values (or expression) of each case (`value1`, `value2`, …). When it finds the matching value, the statements inside that case are executed.

But, if none of the above cases matches the expression, the statements inside `default` block is executed. The default statement at the end of switch is similar to the else block in if else statement.

However a problem with the switch statement is, when the matching value is found, it executes all statements after it until the end of switch block.

To avoid this, we use `break` statement at the end of each case. The break statement stops the program from executing non-matching statements by terminating the execution of switch statement.

Making Decisions Using the switch Statement

- The switch structure uses four new keywords:
  - **switch** starts the structure and is followed immediately by a test expression
  - **case** is followed by one of the possible values that might equal the switch expression
  - **break** optionally terminates a switch structure at the end of each case
  - **default** optionally is used prior to any action that should occur if the test expression does not match any case

Loop

Loops are used to execute a block of code repeatedly based on a condition. They are fundamental in solving real-world problems such as transaction processing, data iteration, and automation.

- C# for loop
- C# while and do...while loop
- Nested Loops in C#: for, while, do-while
- foreach

C# for Loop

A **for loop** is used when the number of iterations is known beforehand.

Syntax

```
for(initialization; condition; increment/decrement)
{
    // code block
}
```

When to Use

- When iteration count is fixed or predictable
- When working with arrays or indexed data
- When you need control over iteration steps

C# for Loop

- The for loop executes a block of code a fixed number of times.
- The three semicolon separated sections of a for statement are used for:
  - Initializing the loop control variable
  - Testing the loop control variable
  - Updating the loop control variable

Real-World Example (Bank Transactions)

```
double[] deposits = {1000, 2000, 1500, 3000};
double total = 0;


for(int i = 0; i < deposits.Length; i++)
{
    total += deposits[i];
}
Console.WriteLine("Total Deposits: " + total);
```

C# while Loop

- A **while loop** executes as long as a condition remains true. It checks the condition *before* execution.

Syntax

```
while(condition)
{
   // code block
}
```

When to Use

- When number of iterations is unknown
- When condition depends on user input or external data
- Continuous processes (e.g., ATM session)

Real-World Example (ATM Withdrawal)

```
double balance = 5000;

while(balance > 0)
{
    Console.WriteLine("Enter withdrawal amount:");
    double amount = Convert.ToDouble(Console.ReadLine());

    if(amount > balance)
    {
        Console.WriteLine("Insufficient funds");
    }
    else
    {
        balance -= amount;
        Console.WriteLine("Remaining Balance: " + balance);
    }
}
```

Do while loop

A **do-while loop** executes at least once, then checks the condition.

Syntax :

```
do
{
    // code block
} while(condition);
```

When to Use

- When code must execute at least once
- Menu-driven systems
- User input validation

Real-World Example (Transaction Confirmation)

```
string confirm;

do
{
    Console.WriteLine("Perform transaction (yes/no):");
    confirm = Console.ReadLine();

    if(confirm == "yes")
    {
        Console.WriteLine("Transaction processed.");
    }

} while(confirm == "yes");
```

C# foreach Loop

- A **foreach loop** is used to iterate through collections (arrays, lists) without using an index.

Syntax

```
foreach(type variable in collection)
{
    // code block
}
```

When to Use

- When working with collections
- When index is not required
- Safer and cleaner iteration

Real-World Example (Bank Statement)

```
double[] transactions = {500, -200, 1000, -150};

foreach(double t in transactions)
{
    Console.WriteLine("Transaction: " + t);
}
```

Comparison

Loop Type	Condition Check	Use Case	Executes At Least Once
for	Before	Known iterations	❌
while	Before	Unknown iterations	❌
do-while	After	Must run once	✅
foreach	Collection	Iteration over data	✅

Question 1: Nested Transaction Analysis

Write a program that:

- Stores multiple customers' transactions (2D array)
- Calculates total balance for each customer
- Stops processing if any transaction is negative beyond -5000

Concepts tested:

- Nested **for loop**
- Conditional break
- Arrays

Question 2: Infinite Loop Debugging

```
int i = 1;
while(i < 10)
{
    Console.WriteLine(i);
}
```

Why does this loop never terminate? Fix it.

Question 3: Foreach Limitation

Why does the following fail?

```
 foreach(int x in arr)
{
    x = x + 10;
}
```

Explain:

- Immutability in foreach
- How to fix using for loop

Question 4: Do-While Edge Case

Predict output:

```
int x = 10;

do
{
    Console.WriteLine(x);
} while(x < 5);
```

Question 5: Real Banking System Logic

Design a system that:

- Keeps asking user for deposit/withdrawal
- Stops only when user types "exit"
- Prevents overdraft

Must use:

- while or do-while
- condition validation
- loop control

Question 6: Trick Question (Mixed Loops)

```
for(int i = 0; i < 3; i++)
{
    int j = 0;
    while(j < 2)
    {
        Console.Write(i + "" + j + " ");
        j++;
    }
}
```

What is the output?

Mid-exam

We will continued here for your Final Exam.

Part 3

Using Arrays

Declaring an Array and Assigning Values to Array Elements

Definition of Array, Purpose of Arrays
Array Declaration and Initialization
Accessing Array Elements Advantages and Limitations

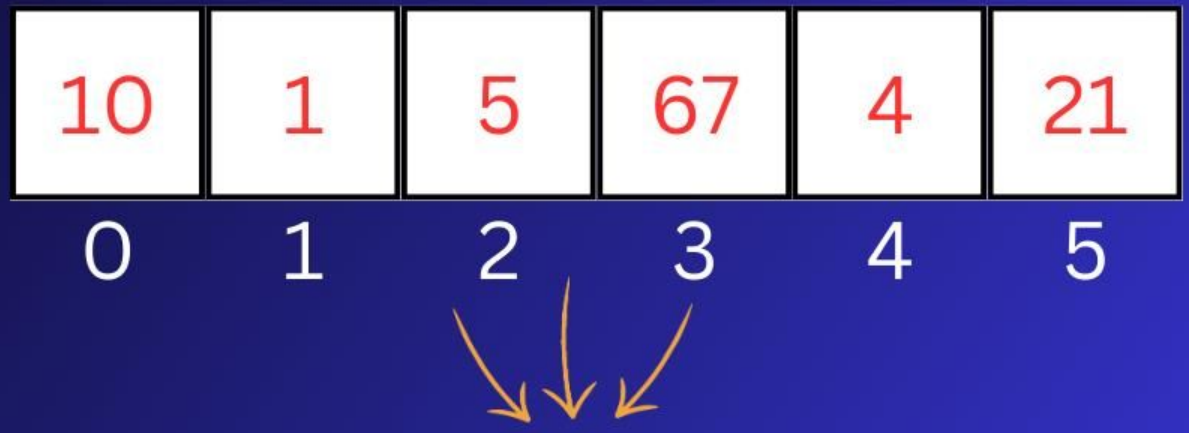
- An **array** is a list of data items that all have the same type and same name
- An array is a data structure used to store multiple values of the same data type in a single variable.
- Arrays are stored in contiguous memory locations.

```
int arr[] = {10, 1, 5, 67, 4, 21}
```

arr[0] = it represents the element at index 0. Similarly, arr[1] represents element at index 1.

array variable name

arr =



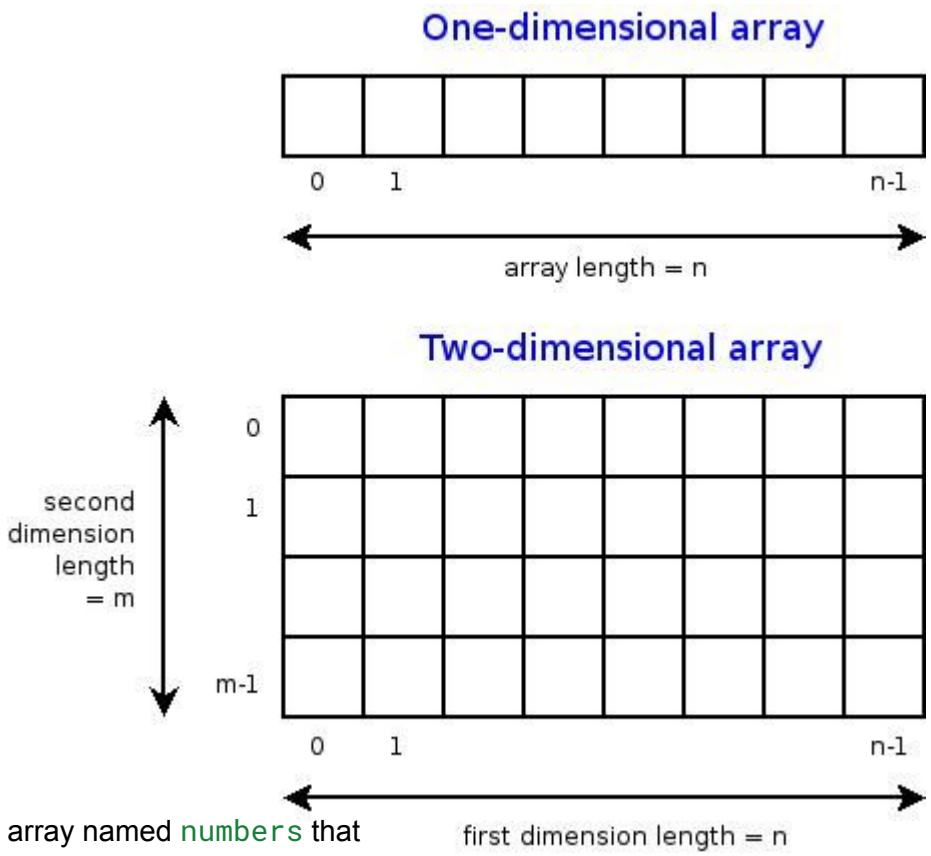
1. Declaring an Array

Syntax First: Declaring and Using an Array (C#)

dataType[] arrayName = new dataType[size];

- **dataType** → The type of values stored (e.g., `int`, `double`, `string`)
- **[]** → Indicates that the variable is an array
- **arrayName** → The name you give to the array
- **new** → Allocates memory for the array
- **size** → Number of elements the array can hold

This syntax both **declares** and **creates** the array in memory.



Example int[] numbers;

This declares an array named `numbers` that will store integers, but it does not yet allocate memory.

2. Creating (Allocating) the Array

To use the array, you must specify its size: example Let's see an example, `int[] age;`

Here, we have created an array named `age`. It can store elements of `int` type. But how many elements can it store?To define the number of elements that an array can hold, we have to allocate memory for the array in C#. For example, `// declare an array`

```
int[] age;

// allocate memory for array

age = new int[5];
```

To use the array, you must specify its size:, Now the array can store 5 integer values, indexed from `0` to `4`.

3. Declaring and Creating Together

int[] numbers = new int[5];

Each element is assigned using its index:

numbers[0] = 10;

numbers[1] = 20;

numbers[2] = 30;

numbers[3] = 40;

numbers[4] = 50;

Shortcut Initialization

You can declare, create, and assign values in one step:

int[] numbers = {10, 20, 30, 40, 50};

An ATM system stores the last 5 transactions of a user.

Step 1: Declare and Create the Array

double[] transactions = new double[5];

This creates an array that can store 5 transaction amounts.

Step 2: Assign Values (Deposits and Withdrawals)

```
transactions[0] = 500.00;   // Deposit
transactions[1] = -200.00;  // Withdrawal
transactions[2] = 1000.00;  // Deposit
transactions[3] = -150.00;  // Withdrawal
transactions[4] = 300.00;   // Deposit
```

Step 3: Access and Display Values

```
for (int i = 0; i <
transactions.Length; i++)
{
    Console.WriteLine("Transaction "
+ i + ": " + transactions[i]);
}
```

Alternative (Shortcut Initialization)

```
double[] transactions = { 500.00,
-200.00, 1000.00, -150.00, 300.00
};
```

Example

- Arrays, like object fields, have default values
- You can assign non default values to array elements upon creation
- Examples:

```
int[] myScores = new int[5] {100,76,88,100,90};// incorrect, why?
int[] myScores = new int[] {100,76,88,100,90};// correct
int[] myScores = {100,76,88,100,90}; // correct
```

Using Subscripts to Access Array Elements

- The power of arrays become apparent when you begin to use subscripts that are variables rather than constant values
- Example:theArray[sub] vs. theArray[1]
- A loop can be used to cycle through the elements of an array
- Through the use of loops and arrays, code can become more efficient

Using the Length Field

- The subscript used to access an array must be between the range of **0 to Length-1**
- Because every array is automatically a member of the class **System.Array**, you can use the fields and methods that are part of the System.Array class
- The **Length()** field is a member of the System.Array class

Using foreach to Control Array Access

- C# supports a **foreach** statement that you can use to cycle through every array element without using subscripts
- With the foreach statement, the programmer provides a temporary variable that automatically holds each array value in turn

Accessing Array Element using foreach

```
using System;

namespace AccessArrayForeach {
  class Program {
    static void Main(string[] args) {
      int[] numbers = {1, 2, 3};

      Console.WriteLine("Array Elements: ");

      foreach(int num in numbers) {
        Console.WriteLine(num);
      }

      Console.ReadLine();
    }
  }
}
```

Multidimensional Arrays

- A multidimensional is an 'array of arrays'.
- It is similar to tables in a database where each primary element (row) is a collection of secondary elements (columns).
- A multidimensional array is an array that contains more than one dimension, meaning it stores data in a table-like structure (rows and columns).
- There are two types of multidimensional arrays in C#:
  - Rectangular array (one in which each row contains an equal number of columns)
  - Jagged array (one in which each row does not necessarily contain an equal number of columns)

dataType[,] arrayName = new dataType[rows, columns];

- **dataType** → Type of elements (e.g., `int`, `double`)
- **[,]** → Indicates a 2D array (two dimensions)
- **rows** → Number of rows
- **columns** → Number of columns
- **arrayName[row, column]** → Access a specific element

```
double[,] transactions = new double[3, 2];
```

```
int[,] transactions = { {1, 2, 3}, {4, 5, 6} };
Console.WriteLine(transactions[1,1]); // Output: 5
```

Assign values

```
transactions[0, 0] = 500;   // Day 1, ATM 1
transactions[0, 1] = 300;   // Day 1, ATM 2

transactions[1, 0] = 700;   // Day 2, ATM 1
transactions[1, 1] = 200;   // Day 2, ATM 2

transactions[2, 0] = 1000;  // Day 3, ATM 1
transactions[2, 1] = 400;   // Day 3, ATM 2
```

Alternative Initialization (Shortcut)

```
double[,] transactions =
{
    {500, 300},
    {700, 200},
    {1000, 400}
};
```

Access and display

```
for (int i = 0; i < 3; i++) // rows (days)
{
    for (int j = 0; j < 2; j++) // columns (ATMs)
    {
        Console.WriteLine("Day " + i + ", ATM " + j + ": " + transactions[i, j]);
    }
}
```

Multidimensional Arrays ([,]) vs Jagged Arrays ([][])

Multidimensional Arrays ([,]) — *True Matrix*

```
dataType[,] arrayName = new dataType[rows, columns];
```

- Stored as a **single rectangular block of memory**
- Every row has the **same number of columns**
- Access using **two indices**: `array[row, column]`

ATM Example (Fixed Structure)

Scenario: 3 days × 2 ATM machines (always consistent)

```
double[,] transactions =
{
   {500, 300},
   {700, 200},
   {1000, 400}
};
```

Access Example

```
Console.WriteLine(transactions[1, 0]); // Day 2, ATM 1 → 700
```

Key Characteristics

- Fixed size (rows × columns)
- Faster for **matrix-like operations**
- Memory is **contiguous**
- Simple and predictable structure

Jagged Arrays ([][]) – Array of Arrays

Syntax

dataType[][] arrayName = new dataType[rows][];

Then initialize each row separately:

arrayName[0] = new dataType[size1];

arrayName[1] = new dataType[size2];

Each row is a **separate array**

Rows can have **different lengths**

Access using: `array[row][column]`

```
double[][] transactions = new double[3][];

transactions[0] = new double[] {500, 300};
// Day 1 → 2 transactions
transactions[1] = new double[] {700};
// Day 2 → 1 transaction
transactions[2] = new double[] {1000, 400, 200};  // Day 3 → 3 transactions
```

ATM Example (Flexible Structure)

Scenario: Each day has different number of transactions

Access Example

Console.WriteLine(transactions[2][1]); // Day 3, second transaction → 400

Core Differences (Clear Comparison)

Feature	Multidimensional (`[,]`)	Jagged (`[][]`)
Structure	Matrix (table)	Array of arrays
Shape	Fixed (rectangular)	Flexible (irregular)
Memory	Contiguous	Non-contiguous
Access	`arr[i, j]`	`arr[i][j]`
Performance	Better for matrix ops	Better for uneven data
Flexibility	Low	High

When to Use Which

Use Multidimensional Array ([,]) when:

- Data is **uniform** (same size everywhere)
- You are working with:
  - Tables
  - Matrices
  - Fixed ATM machines per day

Use Jagged Array ([ ] [ ]) when:

- Data is **irregular**
- Each row has **different number of elements**
- Real-world scenarios:
  - ATM transactions per day (varies)
  - Students taking different number of courses
  - Blockchain blocks with varying transactions

Important Insight (Real Systems)

In real-world systems (banking, Amazon, blockchain):

- Data is **rarely perfectly rectangular**
- Jagged arrays (or dynamic structures like Lists) are more practical

Common Student Mistake

❌ Mixing syntax:

```
int[,] arr = new int[3][];
```

This is invalid because:

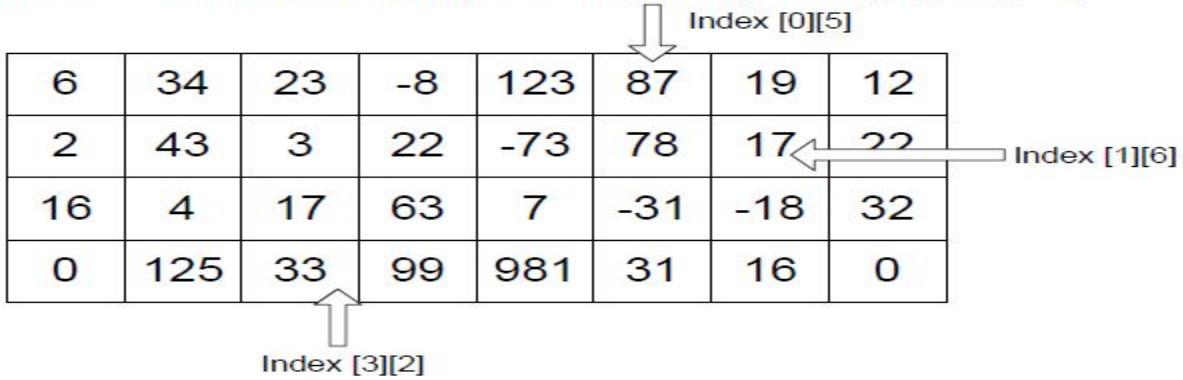
- `[ , ]` and `[ ][ ]` are **different structures**

Final Conceptual View

- `[ , ]` → Think **Excel table (fixed rows & columns)**
- `[ ][ ]` → Think **folders with different number of files inside each**

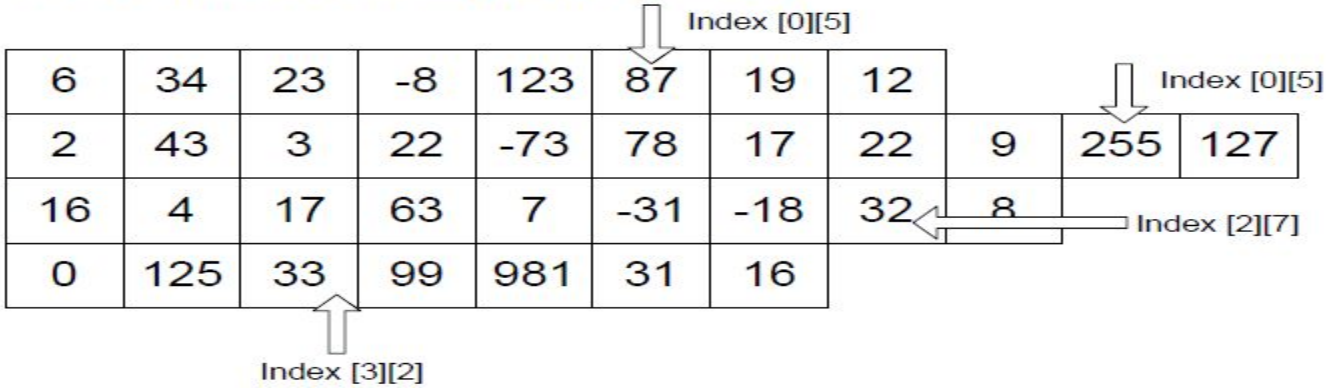
Write a C# implementation for the following array

A two-dimensional (rectangular) Array (4 by 4)



6	34	23	-8	123	87	19	12
2	43	3	22	-73	78	17	22
16	4	17	63	7	-31	-18	32
0	125	33	99	981	31	16	0

A two-dimensional (jagged) Array



6	34	23	-8	123	87	19	12			
2	43	3	22	-73	78	17	22	9	255	127
16	4	17	63	7	-31	-18	32	8		
0	125	33	99	981	31	16				

Using Parameter Arrays

- When you don’t know how many arguments you might eventually send to a method, you can declare a local array within the method header using the keyword **param**
- For example:

  public static void DisplayStrings (param string[] people)

Exercise

Debugging Lab Exercises (C# Arrays)

Focus: Multidimensional ([,]) and Jagged ([ ][ ]) Arrays

```
int[,] arr = {
    {10, 20},
    {30, 40}
};

for (int i = 0; i <= 2; i++)
{
    for (int j = 0; j <= 2; j++)
    {
        Console.WriteLine(arr[i, j]);
    }
}
```

- Identify the bug and fix

```
int[][] arr = new int[2][];
arr[0] = new int[] {1, 2};
Console.WriteLine(arr[1][0]);
```

Why does this crash?
Fix the program

```
int[][] arr = new int[2, 2];
```

- Explain why this is invalid
- Rewrite it correctly as:
  - Multidimensional array
  - Jagged array

```
int[][] arr = {
    new int[] {10, 20},
    new int[] {30}
};

Console.WriteLine(arr[1][1]);
```

Identify the issue
Fix it without changing the data meaning

```
int[,] arr = new int[2,2];
arr[0] = 5;
```

Why is this invalid?
Correct the assignment

Exercise

```
int[][] arr = {
    new int[] {1, 2},
    new int[] {3, 4, 5}
};

for (int i = 0; i < arr.Length; i++)
{
    for (int j = 0; j < arr.Length; j++)
    {
        Console.WriteLine(arr[i][j]);
    }
}
```

- Find the logical bug
- Fix the inner loop

Exercise

```
double[][] transactions = new double[3][];

transactions[0] = new double[] {500, 200};
transactions[1] = new double[] {300};

for (int i = 0; i < transactions.Length; i++)
{
    for (int j = 0; j < transactions[i].Length; j++)
    {
        Console.WriteLine(transactions[i][j]);
    }
}
```

- Identify the hidden bug
- Fix it properly

passing array as method parameter

OOP Concept

Chapter

Basic OOP concepts

C# OOP(I)

- C# Class and Object
- C# Method
- C# Access Modifiers
- C# Variable Scope
- C# Constructor
- C# this Keyword
- C# Destructor
- C# static Keyword

C# OOP(II)

- C# Inheritance
- C# abstract class and method
- C# Nested Class
- C# Partial Class and Partial Method
- C# sealed class and method
- C# interface
- C# Polymorphism
- C# Method Overloading
- C# Constructor Overloading

C# Class and Object

- C# is an object-oriented program.
- In object-oriented programming(OOP), we solve complex problems by dividing them into objects.
- To work with objects, we need to perform the following activities:
  - create a class, How?
  - create objects from the class, How? Suppose use ATM Machine

C# Class

- Class is the blueprint for the object.
- We can think of the class as a sketch (prototype) of a house.
- It contains all the details about the floors, doors, windows, etc.
- We can build a house based on these descriptions.
    - House is the object.
- Like many houses can be made from the sketch, we can create many

Create a class in C#

- We use the class keyword to create an object. For example,
- class ClassName { }
- Here, we have created a class named ClassName. A class can contain
- fields - variables to store data (Properties, Attributes)
- methods - functions to perform specific tasks (Behaviours)

Example

```
class Dog {
  //field
  string breed;
  //method
  public void bark() { }}
```

Here

- `Dog` - **class name**
- `breed` - **field**
- `bark()` - **method**

Note: In C#, fields and methods inside a class are called members of a class.

C# Objects

- An object is an instance of a class. Suppose, we have a class Dog. Bulldog, German Shepherd, Pug are objects of the class.
- Creating an Object of a class
- In C#, here's how we create an object of the class.
- ClassName obj = new ClassName();
- Here, we have used the new keyword to create an object of the class. And, obj is the name of the object. Now, let us create an object from the Dog class.

```
Dog bullDog = new Dog();
```

Now, the `bullDog` object can access the fields and methods of the `Dog` class

Access Class Members Using Object

- We use the name of objects along with the . operator to access members of a class. For example,

```
1.  using System;
2.  namespace ClassObject {
3.    class Dog {
4.      string breed;
5.      public void bark() {
6.        Console.WriteLine("Bark Bark !!");
7.      }
```

Access Class Members Using Object

```
1.  static void Main(string[] args) {
2.      // create Dog object
3.      Dog bullDog = new Dog(); // access breed of the Dog
4.      bullDog.breed = "Bull Dog";
5.      Console.WriteLine(bullDog.breed); // access method of the Dog
6.      bullDog.bark();
7.      Console.ReadLine();}}}
```

Output?

Creating Multiple Objects of a Class

```
using System;
namespace ClassObject
{
    class Employee  {
        string department;
        static void Main(string[] args){
            // create Employee object
            Employee sisay = new Employee();
            // set department for sisay
            sisay.department = "Software
Engineering";
            Console.WriteLine("Sisay: " +
sisay.department);
```

Creating Multiple Objects of a Class

```
// create second object of Employee
  Employee negash = new Employee();
  // set department for negash
  negash.department = "Software Development”;
  Console.WriteLine("Negash: " + negash.department);
  Console.ReadLine();  }}}
```

Here How many objects we created ? what are they?:from the class named Employee.

Output ?

Creating objects in a different Class

In C#, we can also create an object of a class in another class. For example,

```
using System;

namespace ClassObject {

  class Employee {
    public string name;
    public void work(string work) {
      Console.WriteLine("Work: " +
work);

    }
  }
```

```
class EmployeeDrive {
   static void Main(string[] args) {
     // create Employee object
     Employee e1= new Employee();
     Console.WriteLine("Employee 1");
     // set name of the Employee
     e1.name="Gloria";
     Console.WriteLine("Name: " + e1.name);
     //call method of the Employee
     e1.work("Coding");
     Console.ReadLine(); }
  }
}
```

Creating objects in a different Class

- In the above example, we have two classes: **Employee** and **EmployeeDrive**.
- Here, we are creating an object **e1** of the Employee class in the EmployeeDrive class.
- We have used the e1 object to access the members of the Employee class from EmployeeDrive. This is possible because the members in the Employee class are public.
- Here, public is an access specifier that means the class members are accessible from any other classes.
- What are Access Modifiers? List some of these and their purpose?

Why Objects and Classes?

- Objects and classes help us to divide a large project into smaller sub-problems.
- Suppose you want to create a game that has hundreds of enemies and each of them has fields like health, ammo and methods like shoot() and run().
- With OOP we can create a single Enemy class with required fields and methods. Then, we can create multiple enemy objects from it.
- Each of the enemy objects will have its own version of health and ammo fields. And, they can use the common shoot() and run() methods.
- Now, instead of thinking of projects in terms of variables and methods, we can think of them in terms of objects.
- This helps to manage complexity as well as make our code reusable.

C# Methods

C# Methods

A method is a block of code that performs a specific task. Suppose you need to create a program for an ATM system. You can create different methods to handle various operations:

- a method to withdraw money
- a method to deposit money

Dividing a complex problem into smaller chunks makes your program easier to understand, maintain, and reuse.

Declaring a Method in C#

Here's the syntax to declare a method in C#.

```
returnType methodName() {
  // method body
}
```

- **returnType** - It specifies what type of value a method returns. For example, if a method has an int return type then it returns an int value.
- If the method does not return a value, its return type is void.
- **methodName** - It is an identifier that is used to refer to the particular method in a program.
- **method body** - It includes the programming statements that are used to perform some tasks. The method body is enclosed inside the curly braces { }

Declaring a Method in C#

Let's see an example in an ATM system:

```
void withdrawMoney() {
    // code
}

void depositMoney() {
    // code
}

void checkBalance() {
    // code
}

void displayMenu() {
    // code
}
```

- Here, the names of the methods are withdrawMoney(), depositMoney(), checkBalance(), and displayMenu(). All of them have a return type of void, meaning they perform actions but do not return any value.

Declaring a Method in C#

```
double withdrawMoney(double balance, double amount) {
    // code
    return balance - amount;
}
double depositMoney(double balance, double amount) {
    // code
    return balance + amount;
}
```

```
double checkBalance(double
balance) {
    return balance;
}
void displayMenu() {
    // code
}
```

Declaring a Method in C#

withdrawMoney() returns a double (updated balance)

depositMoney() returns a double (updated balance)

checkBalance() returns the current balance

displayMenu() uses void because it only displays information

Calling a Method in C#